A Multi-Branch AI System for SQL Injection Detection at the Database Proxy with a Zero-Day Generalization Study

1st Bach Luong-Chi
School of Science, Engineering & Technology
RMIT University Vietnam
Hanoi, Vietnam
chibachluong@gmail.com

2nd Minh-Duc Do-Xuan
International School
Vietnam National University Hanoi
Hanoi, Vietnam
minhdc42195@gmail.com

3rd Diep Dinh-Ngoc
International School
Vietnam National University Hanoi
Hanoi, Vietnam
dinhngocdiep281104.work@gmail.com

4th Minh Nguyen-Quang
International School
Vietnam National University Hanoi
Hanoi, Vietnam
quangminhdang1703@gmail.com

5th Linh Dinh-Van
[TODO: dept]
Academy of Cryptography Techniques
Hanoi, Vietnam
vanlinh@actvn.edu.vn

6th DinhThai Kim
International School
Vietnam National University Hanoi
Hanoi, Vietnam
thaikd@vnu.edu.vn

Abstract—SQL Injection (SQLi) remains a damaging web application vulnerability. Signature-based Web Application Firewalls (WAFs) struggle with obfuscated payloads and previously unseen attacks. This paper presents a multi-branch artificial intelligence system for SQLi detection. We position the system at the database proxy to inspect the final SQL statement after application construction but before database execution. The architecture combines a supervised multi-class classifier (Branch 1) and a query-level anomaly detector (Branch 2). Branch 2 trains exclusively on benign traffic. We introduce the Session Correlator to catch blind and multi-step attacks. This mechanism reuses signals from both branches without additional training. The evaluation combines four public datasets spanning structurally distinct traffic; a leave-one-out ablation confirms each source is necessary, since dropping any one leaves its own held-out benign traffic with a 65-80 percent false-positive rate. Branch 1 (DistilBERT) reaches a 0.9699 F1-macro across six SQLi categories, including a real stacked-query class previously represented only by synthetic templates, chosen over a TF-IDF plus Logistic Regression baseline for markedly better accuracy on the two hardest classes at a latency that remains real-time-compatible. Branch 2 uses Local Outlier Factor (LOF) to attain a 0.915 Area Under the Curve (AUC), detecting 78.4 percent of anomalous queries at a 5.3 percent false-positive rate (FPR). A zero-day study shows supervised classification remains robust for distinctive attacks but misses Boolean-blind payloads at an 87 percent rate. The anomaly branch independently recovers most of these on structural grounds alone. The Session Correlator recovers residual cases on a held-out set of self-generated attack sessions with zero observed false positives. We report this as preliminary evidence pending validation on independently-captured attacker traffic. Finally, we evaluate an offline continual-learning loop using drift monitoring and a review queue to surface a new class and trigger automatic retraining; a validation gate promotes the resulting major-bump model directly, then correctly rejects a subsequent minor-bump candidate with higher F1 but higher FPR before it reaches shadow, evidence the gate weighs more than a single metric. These results quantify the limits of query-level detection and motivate session-level correlation and disciplined retraining as low-cost mitigations.

Index Terms—SQL injection, intrusion detection, anomaly detection, machine learning, web security, zero-day detection

I. Introduction

Web applications store increasingly sensitive data, making their back-end databases a persistent target. SQL Injection (SQLi), in which unsanitized user input is concatenated into a SQL statement, remains one of the oldest and most severe such threats, ranked among the OWASP Top 10 web application risks [12]. Successful exploitation can disclose, alter, or destroy data and escalate privileges. Although Web Application Firewalls (WAFs) are widely deployed, rule-based defenses depend on predefined signatures; obfuscated payloads and zero-day techniques bypass them, while overly strict rules raise the false-positive rate (FPR) on legitimate traffic.

Machine learning (ML) and deep learning have emerged as promising alternatives because they learn attack patterns from data rather than from hand-written rules [1]–[3]. However, most published AI-based SQLi detectors classify a single query in isolation. This is a structural limitation: Boolean-blind and time-blind SQLi, and query-

splitting attacks, distribute their malicious intent across many individually innocuous-looking queries within one session. Their signal only becomes visible when a sequence of related queries is analyzed together.

This paper describes a multi-branch AI system that performs SQLi detection at the database proxy, inspecting the fully-built SQL statement before it reaches the database server. Two query-level branches, a supervised multi-class classifier and a benign-only anomaly detector, feed a session-level correlation mechanism that catches blind and multi-step attacks neither branch sees alone. On a combined public dataset, the system reaches an F1-macro of 0.9699 for query-level classification and an AUC of 0.915 for anomaly detection, and a leave-one-out study shows the two branches together recover most attacks that either one misses individually. The contributions of this paper are:

- A multi-branch detector architecture combining a supervised multi-class classifier (Branch 1) and a benign-only anomaly detector (Branch 2) under a single decision policy, with a session-level correlation mechanism (the Session Correlator) that reuses both branches' existing signals, with no additional training, to catch blind and multi-step SQLi; evaluated with a mandatory ablation over its two constituent checks.
- A leave-one-out zero-day study that quantifies, per attack category, how far each branch generalizes to an attack type it never saw during training, and what this implies for the value of the anomaly branch.
- An offline evaluation of a continual-learning loop - drift monitoring, a threshold-triggered review queue, and a validation gate with a shadow stage - using a real held-out attack class, with an ablation isolating the retrain gain to the class itself rather than to added data volume.

The remainder of the paper reviews related work (Section II), describes the proposed system (Section III) and the experimental setup (Section IV), reports results (Section V), discusses limitations (Section VI), and concludes (Section VII).

II. Related Work

Traditional SQLi defenses rely on input validation, parameterized queries, and rule-based WAFs such as ModSecurity with the OWASP Core Rule Set [13]. These are fast, interpretable for known attacks, and need no training data; their weakness is the flip side of that same strength, since a fixed rule set cannot anticipate a payload it has never seen, so obfuscated or novel injections bypass it while overly strict rules raise false positives on legitimate traffic [1], [13]. This trade-off, not a failure specific to any one rule set, is what motivates learning-based detection below.

ML-based detectors extract features from queries, including bag-of-words, Term Frequency-Inverse Document Frequency (TF-IDF), or character/word n-grams, and train classifiers such as SVM, Random Forest, or gradient-boosted trees [2]. Deep models remove the need for hand-crafted features. CNNs capture local token patterns [3], while recurrent models (LSTM/GRU) and Transformer-based encoders such as DistilBERT model longer-range structure [5]–[8]. For unseen attacks, anomaly and ensemble approaches have been proposed, including autoencoder/LSTM/GRU ensembles for zero-day web attacks [4]. Classic one-class methods, including Isolation Forest [9] and One-Class SVM [10], model the support of the benign distribution and flag deviations, and adversarial tools such as WAF-A-MoLE [11] expose the fragility of detectors under deliberate evasion. Recent work continues to report competitive results from classical ML with careful feature selection [14], [15], alongside the deep-learning direction this paper also pursues.

Across this literature, detection is predominantly query-level, issuing a verdict per query. Session-level modeling, which is required to expose blind SQLi spread over many requests, is comparatively underexplored. Our work targets that gap while grounding the design in a measured evaluation of the two query-level branches it builds on.

III. Proposed System

A. System Placement: The Database Proxy

The detector is placed at "Position B": it receives the SQL statement after the application has finished constructing it but before the statement reaches the database. Inspecting the final statement (rather than raw HTTP input) removes ambiguity introduced by application-layer transformations and yields a canonical view of what will actually execute. Fig. 1 shows the overall flow: each query is canonicalized and scored by Branch 1 and Branch 2; the Session Correlator correlates both branches' signals over a whole session; the decision engine emits ALLOW/BLOCK/OVERKILL. Algorithm 1 summarizes the per-query decision.

Algorithm 1 Multi-branch detection at Position B (per query)

Require: query q, models B_1, B_2, correlator S
$q_c \leftarrow \text{canonicalize}(q)$
$(p, y_1) \leftarrow B_1.\text{predict}(q_c)$ $\{p = 1 - P(\text{normal})\}$
$s \leftarrow B_2.\text{score}(q_c)$
$is_sqli \leftarrow (p \geq 0.5)$; $is_anom \leftarrow (s > \tau_q)$
if S.is_ready and S.score_session($[q]$) then return BLOCK
if is_sqli then return BLOCK
if $\neg is_sqli \wedge is_anom$ then return OVERKILL
return ALLOW

B. Canonicalization

To reduce syntactic evasion, each statement is normalized before feature extraction: common encodings (URL, hex, CHAR()/ASCII()) are decoded, SQL keyword casing is normalized, and comment sequences (/* */, –) are marked as a feature rather than deleted. The goal is to collapse equivalent syntactic variants so that trivial obfuscation does not change the model's input.

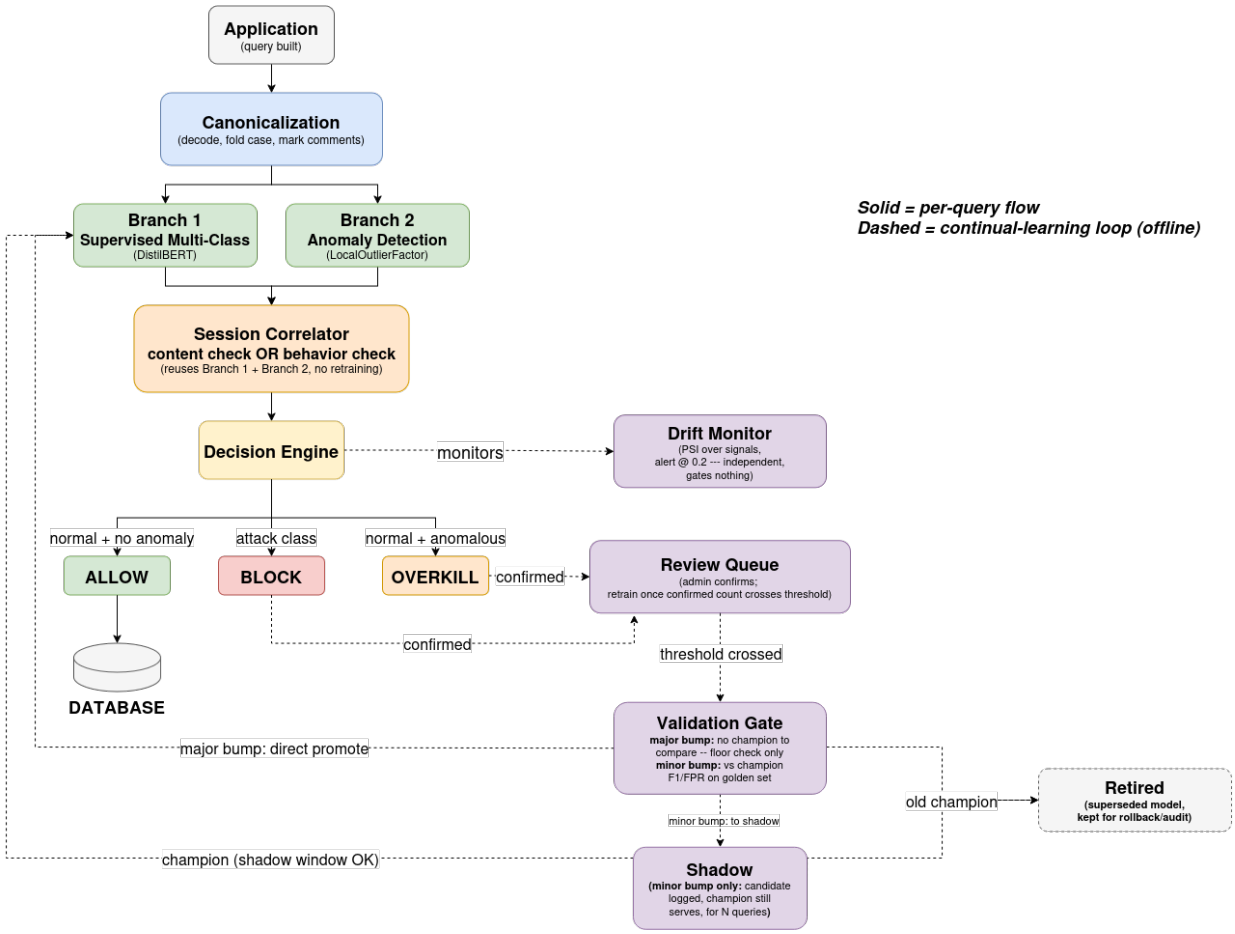


Fig. 1. Multi-branch detector at the database proxy.

C. Branch 1 - Supervised Multi-Class Classification

Branch 1 assigns each query to one of six classes: normal, union_based, error_based, boolean_blind, time_blind, and stacked. We compare four candidate architectures (Section V) and select a fine-tuned DistilBERT classifier for its accuracy on the two hardest classes at a latency that remains real-time-compatible. Multi-class (rather than binary) output lets the decision engine and human administrators reason about which attack technique is detected.

D. Branch 2 - Query-Level Anomaly Detection

Branch 2 is trained on benign traffic only, learning the "normal region" of query behavior and emitting a continuous anomaly score per query, from twelve lightweight features: four whole-string statistics (length, special-character ratio, SQL-keyword count, Shannon entropy), a quote-imbalance count, and six "local-peak" features (longest same-type token run, max token length, token count, longest special-character run, longest digit run, parenthesis imbalance) chosen so long parameter strings from HTTP-derived sources do not dilute the signal. A global One-Class SVM boundary misclassified benign traffic from one source as anomalous merely for not resembling another's; a per-source ensemble was tried next and rejected for an unacceptably high FPR ($\approx$41%). LocalOutlierFactor was adopted instead: its local-density notion judges each query against its own neighborhood rather than one global boundary, better accommodating a benign pool spanning structurally distinct sources. Training without attack labels makes the branch, in principle, sensitive to attack shapes never seen by Branch 1, the property probed in the zero-day study; its score also feeds the Session Correlator.

E. Session Correlator (Branch 3) - Session-Level Correlation

The Session Correlator targets two attack patterns invisible to any query-level branch, since each step looks individually unremarkable: (1) blind SQLi (boolean-blind, time-blind), where an attacker recovers data one bit at a time via a bisection search across many queries, and (2) query-splitting, where a single malicious statement is fragmented across consecutive requests. Our first design, a Gated Recurrent Unit (GRU) consuming Branch 1's classifier output concatenated with Branch 2's anomaly score

per step, fails a diagnostic check: two consecutive Boolean-blind bisection steps differing only in comparison bound have TF-IDF cosine similarity 0.961 at the vectorizer stage but near-identical probabilities after Branch 1's classifier, a lossy projection trained to separate attack type, not preserve content - the signal a sequence model needs is collapsed before it arrives. We therefore replaced the GRU with the Session Correlator: not a trained model, but two independent checks over the already-trained Branch 1 and Branch 2, OR'd together. The content check concatenates a session's raw query text and re-scores it with Branch 1's classifier against a threshold calibrated for session-length text. The behavior check aggregates Branch 2's per-query anomaly scores (median, fraction above a calibrated threshold), run exactly as trained, so no train/inference shift is introduced. Both checks are calibrated on a disjoint TRAIN split and evaluated on a held-out TEST split (Section V); each covers cases the other misses, leaving one residual gap by design: an attacker evading both checks simultaneously, which neither branch's signal can close. Because the blind-SQLi session generator bisects toward a fixed target value, its target pool is partitioned into disjoint train and test subsets before session generation, so the held-out evaluation measures generalization to unseen targets rather than recall of seen ones.

Let $S = [q_1, \ldots, q_n]$ be a session consisting of n chronologically ordered queries from the same client. Denote $p(S) = 1 - P_{B_1}(\text{normal} \mid \|_i q_i)$ for the concatenated text and $s_i = B_2.\text{score}(q_i)$ per query. With calibrated thresholds $\tau_c, \tau_q, \tau_{med}, \tau_{frac}$, the checks are defined as:

$$\begin{aligned} fires_c(S) &= \mathbb{I}[p(S) \geq \tau_c], \\ frac(S) &= \frac{1}{n}\sum_{i=1}^{n} \mathbb{I}[s_i > \tau_q], \\ fires_b(S) &= \mathbb{I}[m(S) \geq \tau_{med}] \vee \mathbb{I}[frac(S) \geq \tau_{frac}] \end{aligned} \tag{1}$$

where $m(S) = \text{median}(s_1, \ldots, s_n)$ and the session verdict is

$fires_c(S) \vee fires_b(S)$. Implemented operationally as in Algorithm 2.

Algorithm 2 SessionCorrelator: calibration and scoring (no gradient training)

```
Calibration (on TRAIN sessions only):
τ_q ← 95th percentile of
    {B_2.score(q) | q ∈ benign TRAIN}
τ_c ← arg max_τ (TPR − FPR) on
    {concat(S_train) scored by B_1}
τ_med, τ_frac ← sweep median and fraction-above-τ_q
    on TRAIN
Scoring a session S = [q_1..q_n]:
a_c ← 1 − P_{B_1}(normal | concat(q_1..q_n)); fires_c ← (a_c ≥ τ_c)
scores ← [B_2.score(q_i) ∀i]; m ← median(scores); frac ← |{s > τ_q}|/n
fires_b ← (m ≥ τ_med) ∨ (frac ≥ τ_frac)
return fires_c ∨ fires_b {content check OR behavior check}
```

F. Decision Engine and Overkill Policy

A central decision engine combines the branch outputs into one verdict, exactly as Algorithm 1 shows: any query classified by Branch 1 as an attack class is blocked; a query classified normal but flagged anomalous by Branch 2 is held as an Overkill case, not executed and queued for administrator confirmation, defaulting to deny on timeout. Confirmed decisions feed a continual-learning loop (proposed). This policy trades a small amount of friction on rare suspicious-but-benign queries for defense-in-depth against zero-days.

G. Continual-Learning Design

Confirmed Overkill/Block decisions accumulate into a labeled pool that a continual-learning loop consumes, so the deployed model can improve without full retraining. Drift monitoring tracks the Population Stability Index (PSI) of five feature and confidence signals against a fixed reference window, alerting at PSI ≥ 0.2, independently of the mechanism below. A review queue holds low-confidence and anomalous queries for administrator confirmation; a volume-threshold policy triggers a retrain automatically once confirmed samples cross a fixed count, rather than on a fixed schedule. A validation gate then decides whether the candidate may replace the deployed model, distinguishing a major bump (the label space changes) from a minor one (same labels, more volume): a major bump has no prior model to compare against, so it is checked instead against an absolute floor (no regression on known classes, a minimum recall on the new one) and promoted directly if it clears that floor; a minor bump must match or exceed the current champion's F1/FPR on the frozen benchmark. Only a promoted minor-bump candidate then enters shadow, logging predictions against further traffic while the incumbent keeps serving, becoming champion only if shadow-observed FPR and F1 stay within tolerance of the gate-measured values. A major-bump candidate skips shadow, since no incumbent exists for that class and the floor check already carries that weight. Algorithm 3 summarizes the loop. Every promotion retires the model it replaces (Shadow → Champion → Retired), with retired models kept for rollback and audit. Section V evaluates this loop as a full offline experiment.

IV. Dataset and Experimental Setup

A. Data Sources

Four public datasets are combined: SQLiV3 (D1, a Kaggle SQLi query collection; license provenance unclear); CSIC 2010 (D2, HTTP traffic, used for benign enrichment and, held out, as a labeled anomalous set); Superviz25 (D3, a ≈3.69M-row Kaggle dataset of real sqlmap runs against a seeded relational schema; license also unverified); and SR-BH 2020 (D4, a CC0 honeypot dataset of 527,813 rows, 250,285 originally tagged SQL Injection) (Section VI). D4's labels are coarse and multi-attack (Common

Algorithm 3 Continual-learning loop (offline, Branch 1)

```
while streaming do
    monitor PSI on 5 signals vs. reference; alert if PSI ≥ 0.2
      for 2 sustained windows {drift, does not gate}
    enqueue low-confidence and anomalous queries for admin review
    if |confirmed| ≥ 500 then
        assemble D' ← D_train ∪ confirmed
          (dedup by query_canonical)
        seal data version v = infer_bump(v_old, v_new)
        if v is major then
            check floor: no known-class recall drop > 0.02
              and new-class recall ≥ 0.80; promote directly if pass
        else
            gate: F1_cand ≥ F1_champ ∧ FPR_cand ≤ FPR_champ
              {shadow 2000 queries if pass}
        end if
    end if
end while
```

Attack Pattern Enumeration and Classification (CAPEC)-tagged), so rows are re-tagged into SQLi sub-types by rule-based matching; D3's own attack_technique column is instead trusted directly, since each row is generated for a known technique against a real database, so its ground truth comes from construction, and a majority are also obfuscated by a real WAF-evasion transform (character-case randomization, whitespace-to-comment substitution) that can break a hand-written regex tagger. Recent SQLi detection papers typically train on tens of thousands to a few hundred thousand rows scraped from Kaggle collections or CSIC 2010 alone [14], [15]; D3 alone, at ≈3.69M rows, is well over an order of magnitude larger, and CSIC 2010, though dated to 2010, remains in active use in 2025 work [15], which is why it is retained here alongside newer sources rather than replaced outright.

These four sources are chosen to cover structurally distinct traffic, a claim checked rather than assumed. A leave-one-out ablation retrains Branch 2's benign pool with each source held out in turn, evaluating FPR against every source's own held-out traffic, including the excluded one (Table IV). Excluding any single source leaves that source's own FPR at 65–80%, while the other three stay in the 2–7% range the full model achieves overall, direct evidence that no source is redundant. An earlier pass of this same ablation, which measured FPR only against each reduced combination's own held-out set, had suggested D1 and D2 could be dropped at negligible cost; that conclusion did not survive testing against the excluded sources' own traffic, an artifact of the narrower test rather than a real property of the data, and was not used to guide the final data composition.

B. Branch 1 Dataset

D3 supplies real stacked-query examples in addition to real examples of the other four attack sub-types (the continual-learning experiment in Section V separately holds out a real class, union_based, from its own training pool, unrelated to stacked).

Each D3 attack row carries an attack_id identifying its multi-step sqlmap campaign; rows from the same campaign are near-duplicates, differing mainly in a narrowing bisection bound, so a plain per-row split would leak near-duplicate content across train/test. D3's attack rows are therefore split by whole campaign, every row sharing an attack_id landing on the same side, while D1/D2/D4 keep the original per-row stratified split. D3's campaigns further split into a recon stage (probing and setup queries, lexically close to legitimate traffic) and an exploit stage (queries carrying the payload); Branch 1 detects exploit-stage rows at 100.0% but recon-stage rows at only 94.0%. Because campaign-level splitting means a class's test set can draw disproportionately from a handful of recon-heavy campaigns, per-class recall for the classes D3 dominates (stacked, time_blind) is sensitive to which campaigns land in the held-out split (Section V).

Content-based signature filtering removes mislabeled attacks from the normal pool over three iterative rounds, and query_canonical text is deduplicated before splitting, so no query text appears verbatim in both partitions. The final dataset is 90,000 rows over six classes (each undersampled to 15,000, seed = 42), split 71,705 train and 18,295 test. F1-macro is the headline metric since accuracy would be distorted by class imbalance.

C. Branch 2 Dataset

Branch 2 pools clean benign rows from all four sources, using a broader content filter that rejects any attack-like traffic. D2 and D4 are captured as full HTTP requests, not raw SQL; since the system consumes only the query/parameter text an application would build (Section III), each such row is stripped to its query-string/body parameters, dropping rows with nothing left after stripping. D3 is already query-shaped, but its benign rows are legitimate multi-table and analytical SQL (joins, sub-queries, geographic functions) against a real schema, structurally unlike the simple parameter lookups the other three sources provide, and, before adding D3's benign traffic, the source of a 69–95% FPR on this kind of legitimate-but-unfamiliar traffic in a zero-shot check run to decide whether adding D3 was warranted. After filtering, de-duplication, and stripping, the benign pool contains 72,162 rows (57,321 train and 14,841 test); a held-out set of 370,002 confirmed-SQLi rows spanning all four sources is used only for evaluation.

D. Session Correlator Dataset

No public dataset provides session-level SQLi labels, so sessions for the two target scenarios are constructed differently, reflecting how each attack unfolds. Blind-SQLi sessions run the same bisection algorithm real SQLi tools use, length probing followed by per-character ASCII probing (∼7 requests per character), against a real, self-hosted SQLite database, so every probe's outcome is a genuine row-count difference or measured SLEEP() delay,

not a templated guess; this is the scenario whose bisection structure exposed the GRU's failure. Query-splitting sessions instead use heuristic token-fragmentation of a single real attack payload across consecutive requests, since no bisection-style search exists for this attack type. Together this yields 1,400 sessions across four classes (benign, boolean-blind, time-blind, query-splitting), split 1,120 training and 280 test, with the 100-user synthetic target pool itself partitioned into disjoint train/test subsets before sessions are generated. Collecting an independent dataset from real sqlmap traffic against a deliberately vulnerable application is future work (Section VII).

E. Evaluation Protocol

Metric choice follows from each branch's task rather than a single convention across all three. Branch 1 is a multi-class classifier over an imbalanced label set, so F1-macro (not accuracy, which the majority classes would inflate) is the headline metric, alongside per-class precision/recall/F1, a confusion matrix, and per-class ROC to show where errors concentrate. Branch 2 is a one-class anomaly detector with no fixed decision boundary a priori, so we report FPR and detection rate at a specific operating point (the deployment-relevant trade-off), AUC (a threshold-independent summary of ranking quality), and a 21-point threshold sweep to make the trade-off curve, not just one point on it, inspectable. The Session Correlator issues a binary per-class flag from two independently OR'd checks, so FPR on benign sessions and detection rate per attack class are reported separately for each check alone and combined - an ablation, not just a combined number, so neither check's contribution is left unmeasured. All experiments are deterministic (seed = 42); latency was measured on a single RTX 3050 (6 GB) workstation.

V. Experimental Results

A. Branch 1 - Supervised Multi-Class

Table I compares four candidate architectures on the six-class dataset (Section IV); Logistic Regression and LightGBM use a TF-IDF representation retuned by grid search for this evasion-heavy data (character n-grams (1,2), $C = 10$, vocabulary shrunk from 50,000 to 5,037 terms), while DistilBERT and the CNN use their own tokenizers. DistilBERT reaches the highest F1-macro (0.965), with LightGBM close behind (0.962) at a similar latency (3.0 ms vs. 2.8 ms): the smaller TF-IDF vocabulary cut LightGBM's latency by $\approx 20\times$, removing the reason it was earlier ruled out for a real-time proxy. The CNN tokenizer (0.749) is the weakest candidate, its small (37K-parameter) capacity not holding up against D3's harder, evasion-heavy attack distribution. Logistic Regression (0.898) is cheapest by two orders of magnitude (0.4 MB, 0.5 ms) but trails DistilBERT by 6.7 points, concentrated in the two classes discussed below; Table II reports the deployed DistilBERT model's own held-out result (F1-macro = 0.9699), independently retrained with the same data and configuration.

TABLE I
Branch 1 architecture comparison (tuned TF-IDF configuration).

Model	F1-macro	p50 lat.	Size	Train
TF-IDF + LogReg	0.898	0.5 ms	0.4 MB	32 s
TF-IDF + LightGBM	0.962	3.0 ms	4.5 MB	110 s
DistilBERT (chosen)	0.965	2.8 ms*	256 MB	1,970 s
CNN + SQL-tokenizer	0.749	0.3 ms	152 KB	13 s

*GPU.

TABLE II
Branch 1 per-class test results ($n = 18{,}295$).

Class	Prec.	Recall	F1	Support
normal	0.890	0.973	0.930	3,000
union_based	1.000	0.994	0.997	3,027
error_based	0.999	0.989	0.994	3,033
boolean_blind	0.990	0.997	0.994	3,016
time_blind	0.984	0.962	0.973	3,007
stacked	0.959	0.905	0.932	3,212

DistilBERT was chosen over the cheaper Logistic Regression because that 6.7-point gap concentrates in stacked and time_blind: 0.700/0.812 F1 for Logistic Regression against 0.932/0.973 for DistilBERT. DistilBERT's 2.8 ms latency remains real-time-compatible, so the accuracy gain is not traded against deployment feasibility; the costs accepted instead are a GPU dependency, a 256 MB artifact (640 times Logistic Regression's), and a roughly 33-minute training time.

Table II gives the per-class breakdown. union_based, error_based, and boolean_blind are robust (F1 $\geq$ 0.99); normal (0.930) and time_blind (0.973) trail slightly, and stacked (0.932) remains hardest, though markedly less so than under Logistic Regression: its test split draws from only 8 of D3's 45 stacked campaigns, a consequence of the campaign-level split required to avoid near-duplicate leakage, and those 8 are recon-heavy, so a small, unlucky campaign sample concentrates the residual error even for a higher-capacity model. Per-class ROC, which measures ranking quality independent of the 0.5 decision threshold, stays high across all six classes (AUC $\geq$ 0.964) even where F1 is lower, so the residual error is specifically between attack sub-types, not between attack and benign. This F1 is measured on a clean test split, not on adversarially perturbed input.

B. Branch 2 - Query-Level Anomaly Detection

Table III compares three anomaly algorithms on the same benign/anomalous pools (Section IV). One-Class SVM under-performs badly now that the benign pool spans four structurally distinct sources; Isolation Forest improves substantially but still leaves most attacks unde-

tected; LocalOutlierFactor was selected for its higher AUC and detection rate, since its local-density notion accommodates a four-source benign pool without one source's traffic looking anomalous merely for not resembling another's. At its chosen operating point (contamination = 0.05), it produces 782 false positives on 14,841 benign queries (FPR = 5.3%) and flags 78.4% of the 370,002 held-out anomalous queries, with average precision = 0.994. Fig. 2 shows the resulting 21-point threshold sweep: the false-positive rate falls sharply over the first few thresholds, while the detection rate declines more gradually across the same range, so the steep initial region trades a small detection loss for a large FPR reduction. Past roughly the tenth threshold, further FPR reduction flattens while detection keeps falling, making that region unattractive; the chosen operating point (contamination = 0.05) sits inside the steep initial region, where the FPR/detection trade-off is most favorable.

TABLE III
Branch 2 algorithm comparison.

Algorithm	Contam.	FPR	Detection	AUC
One-Class SVM	0.001	0.20%	11.08%	0.579
Isolation Forest	0.01	1.22%	47.23%	0.903
LocalOutlierFactor (chosen)	0.05	5.27%	78.42%	0.915

DR = detection rate; FPR = false-positive rate.

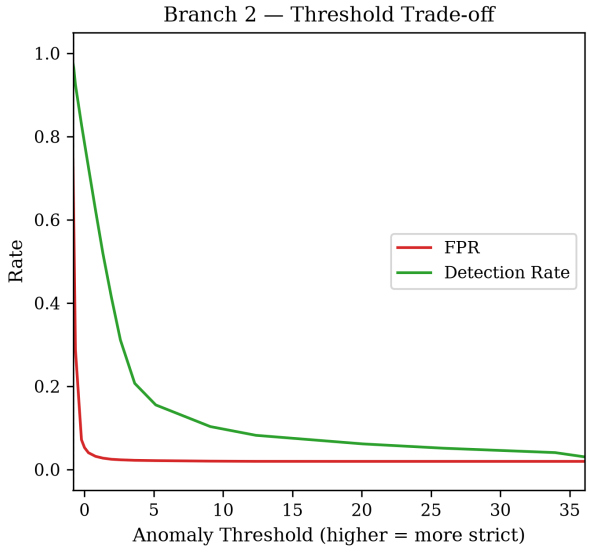


Fig. 2. Branch 2 threshold trade-off: false-positive rate vs. detection rate across 21 operating points.

TABLE IV
Benign-source leave-one-out ablation (bold: FPR on the excluded source).

Trained on	D1 FPR	D2 FPR	D3 FPR	D4 FPR
All four	4.8%	2.8%	7.2%	2.8%
Minus D1	79.0%	2.9%	7.1%	2.7%
Minus D2	4.5%	64.8%	6.8%	2.8%
Minus D3	6.3%	3.8%	72.3%	4.4%
Minus D4	4.0%	2.0%	6.3%	79.8%

C. Zero-Day Generalization Study

To probe robustness to unseen attack categories, we run a leave-one-out protocol over five labels (normal, union_based, error_based, boolean_blind, time_blind): for each attack class, that class is removed and a dedicated TF-IDF + Logistic Regression Branch 1 is retrained on the remaining four labels (chosen over DistilBERT here for this 4-fold retraining loop's much lower per-fold cost); Branch 2, benign-only and never exposed to attack labels, is not retrained per fold. Table V reports the Branch 1 miss rate (fraction predicted normal), the Branch 2 detection rate on the same held-out queries, and their combined coverage (caught by at least one branch).

Supervised classification generalizes well to structurally distinctive unseen classes: withholding union_based, error_based, or time_blind still leaves Branch 1 flagging almost all queries as some attack (miss rate $\leq$2.6%), and the anomaly branch independently catches nearly all of them too (98.3–99.1%), since distinctive structure registers as anomalous regardless of what Branch 1 was trained on. boolean_blind remains the failure mode, though less starkly than either branch alone suggests: Branch 1 misclassifies 86.5% as normal (lexically close to legitimate queries), yet the anomaly branch independently catches 87.8% on structural grounds, and the two combined still leave 11.2% uncaught, the lowest coverage of any class by a wide margin, the empirical motivation for session-level correlation. At the session level this same condition closes entirely: the Session Correlator's behavior check, depending only on Branch 2 and thus unaffected by Branch 1's blindness, flags all 70 held-out boolean_blind sessions (Section V), on a different, self-generated evaluation set, so this is complementary evidence rather than a directly comparable number.

D. Session Correlator

Table VI reports the mandatory ablation on the 280-session held-out self-hosted test set (disjoint train/test target pools, Section III), recalibrated against the deployed DistilBERT Branch 1: each row runs only the named mechanism, except the last, which is how the deployed system runs the two OR'd together. The behavior check's session-level aggregate is each session's median, not mean, per-query Branch 2 score: LocalOutlierFactor's score is

TABLE V
Leave-one-out zero-day results (per unseen class).

Unseen class	B1 miss	B2 detection	Combined
union_based	2.6%	99.1%	99.8%
error_based	0.0%	98.3%	100.0%
boolean_blind	86.5%	87.8%	88.8%
time_blind	0.1%	99.0%	100.0%

TABLE VI
Session Correlator ablation ($n = 280$ held-out sessions).

Configuration	FPR	DR bool.	DR time	DR split
Content check	0/70	1.0	1.0	0.957
Behavior check	0/70	1.0	0.0	0.386
Combined (OR)	0/70	1.0	1.0	0.957

mathematically unbounded, and a handful of structurally degenerate short queries (from query_splitting fragmentation) score many orders of magnitude above the rest, which a raw mean let dominate a session's aggregate, pushing benign-session FPR from 0% to 37% before this fix (Section VI).

Content check and Combined (OR) reach zero observed false positives and near-perfect detection on all three attack classes (query_splitting: 67/70); Behavior check alone does not fully catch query_splitting (0.386) and, with DistilBERT in place, no longer independently catches time_blind (0.0, versus 1.0 under the original TF-IDF calibration), both blind spots the content check closes, which is why the checks are OR'd rather than used alone.

A separate ablation re-scoring the content check on boolean-blind sessions with the leave-one-out Branch 1 variant (never trained on boolean-blind) drops detection to 0.0, mean attack probability stuck around 0.22 regardless of concatenation length: concatenation amplifies a signal Branch 1 already recognizes, so it cannot rescue a classifier that never learned the class.

Reading these numbers narrowly. Zero false positives on 70 benign sessions bounds the true FPR only loosely (a 95% Wilson interval reaches roughly 4%); sessions are drawn from the same generation process Branch 1 and Branch 2 were built around, not independently-captured traffic, though disjoint train/test target pools (Section III) at least test generalization to unseen targets within that process. This is evidence the mechanism works as designed, not yet of generalization to adversarially-diverse real traffic, which future independently-captured attacker data is intended to test.

E. Continual Learning

This experiment uses its own dedicated training and replay pool, reused for all three retraining stages (M0/M1/M2), independent of the main Branch 1 pipeline. union_based is entirely absent from M0's initial training pool (28,688 rows over the remaining five known classes); a frozen golden set (9,000 rows, all six classes) is held out throughout. A two-pour replay stream, padded with real held-out benign traffic, follows: Scenario 1 interleaves union_based into ordinary traffic; Scenario 2 is more known-class volume only, no new class. A review queue and drift monitor watch both pours independently.

On the golden set, M0 reaches F1-macro = 0.9712 (FPR = 0.0073) over the five known classes; on union_based, unseen by M0, Branch 1 alone misses 2.1%, while the fixed Branch 2 independently flags 77.4%, for 99.5% combined coverage. Once 500 confirmed union_based samples accumulate (39.9% through Scenario 1), a retrain triggers automatically; the resulting M1 (F1-macro = 0.9711, FPR = 0.0100, 98.6% recall on union_based) clears the major-bump promotion floor and is promoted directly, later independently recalling 98.7% of Scenario 1's remaining occurrences on fresh stream data. A control retrained on the identical +1,753-row budget without union_based reaches 0.000 recall versus M1's 0.986, confirming the gain is attributable to the class itself, not more data; drift monitoring independently confirms the same shift, crossing its alert threshold by window 3.

With M1 as champion, Scenario 2's confirmed volume produces M2: higher F1 (0.9739) but nearly double the FPR (0.0207 vs. 0.0100). The zero-slack gate rejects M2 on FPR alone ($0.0207 > 0.0100$), so M1 remains champion; an F1-only comparison would have promoted M2 instead. Replayed against M1, Scenario 2's drift signals stay flat, confirming ordinary volume growth alone does not look like drift.

Reading these numbers narrowly. Both pours use a higher attack concentration than production would see, and traffic is a replay of held-out data, not live production; the review queue and gate are tested as a library, not yet wired into the deployed service for all branches. Because M2 was rejected before shadow, this run does not exercise the shadow-validation path, covered instead by this project's own library tests.

VI. Discussion and Limitations

Latency budget. End-to-end per-query latency, estimated from measured per-branch costs rather than live load, is ≈18–21 ms (Branch 1: 2.8 ms on GPU; Branch 2: 15–18 ms for its k-nearest-neighbor lookup, versus a fixed boundary like One-Class SVM), still compatible with a real-time proxy but not yet re-measured against the live /api/v1/detect endpoint or under sustained throughput.

Label noise. Two measured noise sources are found during data construction: mislabeled attacks in the D4 normal pool (requiring three rounds of content filtering) and ≈13% mislabels in the boolean_blind catch-all class (30-sample manual audit, a small sample). Both remain real, secondary contributors to Table II's errors, but the dominant confusion (stacked↔time_blind) is explained

separately in Section V by D3's recon/exploit campaign structure; the clean-label ceiling for this task is unknown.

Dataset licensing. D4 (CC0) has a confirmed license; D1 (SQLiV3) and D3 (Superviz25) both lack an explicit license; D2 (CSIC 2010)'s license was not independently re-verified for this revision. The combined dataset should be treated as provenance-unclear pending resolution.

Threat-model and robustness boundaries. Second-order and out-of-band SQLi, and non-SQL web attacks (Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF)), are out of scope by the proxy placement. All results are on clean test splits; no adversarial evaluation (e.g. WAF-A-MoLE [11]) was run, so the reported figures are an upper bound on robustness to deliberate evasion.

LocalOutlierFactor's costs. Unlike a fixed-boundary model (One-Class SVM, Isolation Forest), LocalOutlierFactor retains its full training feature matrix and scores each query via a k-nearest-neighbor lookup, giving a larger artifact and higher latency (Section V) in exchange for its higher detection rate. Its raw anomaly score is also mathematically unbounded (scores in the billions on a handful of structurally degenerate short queries, versus under 15 for the other two algorithms); this leaves Table III's threshold- and rank-based metrics unaffected, but once silently broke the Session Correlator's behavior check, whose mean-based aggregate a single such outlier could dominate, pushing benign-session FPR from 0% to 37% before switching the aggregate to the median (Section V).

The continual-learning gate's zero FPR slack. The minor-bump path (Section V) requires no FPR regression at all against the champion, unlike the separate shadow-stage tolerance applied only after a candidate has already passed the gate; this is a stricter bar than an F1-only comparison would apply, and one sensitive to how precisely FPR is estimated on a finite golden set.

Session Correlator operational scope. The single-query endpoint currently invokes the Session Correlator with only that one query as a degenerate one-step session; realizing session-level correlation across separate requests over time requires a session store (keyed by session/IP with a TTL) that has not yet been built. The dedicated session endpoint, evaluated in Section V, already accepts a complete session's queries in one call.

VII. Conclusion and Future Work

We presented a multi-branch AI system for SQLi detection at the database proxy, combining a supervised six-class classifier (Branch 1, DistilBERT, F1-macro = 0.9699) with a benign-only anomaly detector (Branch 2, LocalOutlierFactor, AUC = 0.915, 78.4% detection at 5.3% FPR) over four public datasets, each shown individually necessary by a leave-one-out ablation (Table IV). A leave-one-out zero-day study shows supervised detection stays robust for structurally distinctive unseen attacks but bypasses boolean-blind SQLi at an 87% rate, with the two branches combined still leaving an 11% residual gap; this ceiling on query-level detection motivates the Session Correlator, which re-uses both branches' existing signals with no additional training and reaches zero observed false positives with near-perfect detection (95.7–100%) on held-out self-generated sessions, not yet validated on independently-captured attacker traffic. An offline continual-learning loop, evaluated using a real held-out attack class, shows a review queue and drift monitoring both surfacing the new class, automatic volume-triggered retraining, a validation gate promoting the resulting major-bump model after an absolute-floor check, and the same gate correctly rejecting a subsequent minor-bump candidate with higher F1 but higher FPR, evidence the gate weighs more than a single metric.

Future work centers on validating the Session Correlator against real sqlmap traffic from a network-isolated vulnerable lab (DVWA/WebGoat), captured via an intermediate proxy, rather than self-generated sessions. Further items include a production session store for incremental cross-request correlation, wiring the review queue and drift dashboard into the deployed service for all branches, adversarial hardening, and re-measuring latency and throughput against the live endpoint under realistic load.

References

[1] A. Paul, V. Sharma, and O. Olukoya, "SQL injection attack: Detection, prioritization & prevention," J. Inf. Secur. Appl., vol. 85, art. 103871, 2024.

[2] A. E. Widodo and F. F. D. Imaniawan, "Detection of SQL injection, XSS, and command injection attacks in web payloads using SVM, Random Forest, and XGBoost," J. Inf. Syst. Informatics, vol. 8, no. 3, 2026.

[3] A. Luo, W. Huang, and W. Fan, "A CNN-based approach to the detection of SQL injection attacks," in Proc. IEEE/ACIS ICIS, 2019.

[4] V. Babaey and H. R. Faragardi, "Detecting zero-day web attacks with an ensemble of LSTM, GRU, and stacked autoencoders," Computers, vol. 14, no. 6, art. 205, 2025.

[5] A. Vaswani et al., "Attention is all you need," in Adv. Neural Inf. Process. Syst. (NeurIPS), 2017. arXiv:1706.03762.

[6] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT," 2019. arXiv:1910.01108.

[7] Y. Liu and Y. Dai, "Deep learning in cybersecurity: A hybrid BERT–LSTM network for SQL injection attack detection," IET Inf. Secur., vol. 2024, art. 5565950, 2024.

[8] D. Lu, J. Fei, and L. Liu, "A semantic learning-based SQL injection attack detection technology," Electronics, vol. 12, no. 6, art. 1344, 2023.

[9] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in Proc. IEEE ICDM, 2008, pp. 413–422.

[10] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," Neural Comput., vol. 13, no. 7, pp. 1443–1471, 2001.

[11] L. Demetrio, A. Valenza, G. Costa, and G. Lagorio, "WAF-A-MoLE: Evading web application firewalls through adversarial machine learning," in Proc. ACM SAC, 2020. arXiv:2001.01952.

[12] OWASP Foundation, "A03:2021 – Injection," OWASP Top 10:2021, 2021. [Online]. Available: https://owasp.org/Top10/2021/A03_2021-Injection/

[13] M. Al-olaqi, A. Al-gailani, and M. M. H. Rahman, "Comprehensive study of SQL injection attacks mitigation methods and future directions," J. Cyber Secur. Risk Audit., vol. 2025, no. 4, pp. 347–365, 2025.

[14] C.-M. Roşca, A. Stancu, and C. Popescu, "Machine learning models for SQL injection detection," Electronics, vol. 14, no. 17, art. 3420, 2025.
[15] E. Casmiry, N. Mduma, and R. Sinde, "Enhanced SQL injection detection using chi-square feature selection and machine learning classifiers," Front. Big Data, vol. 8, art. 1686479, 2025.